MATERIAL PARA ASISTENTES

Agentes de IA con Pydantic AI

Guía de la charla: lo que se explicó, por qué importa y cómo seguir aprendiendo.

Preparado para
PyCon Panamá 2026 · Python intermedio

Fecha y hora
Viernes 23 de octubre de 2026

Preparado por
Ricardo Tovar

01 — De qué trata esta guía

Esta guía acompaña la charla «Agentes de IA con Pydantic AI: de cero a producción con menos magia y más ingeniería». Reúne lo que se explicó en la sala, con más detalle del que cabe en una diapositiva, y propone un camino para seguir aprendiendo después.

La tesis de la charla cabe en una frase: **un agente útil no es solo un modelo bueno. Es un modelo dentro de un sistema que lo vuelve controlable.** Ese sistema se construye con piezas conocidas de Python: tipos, validación, funciones, un servicio web, límites y trazas.

Para quién está escrita: personas con Python intermedio que quieren pasar de un experimento con un modelo de lenguaje a un servicio que otras personas puedan usar con confianza. No hace falta experiencia previa con agentes.

02 — Lo que se vio en la charla

Parada	Qué se explicó
Agent loop	Cómo decide un agente
Stack y código	Un agente con FastAPI: contrato, agente, herramienta y endpoint
Pydantic AI	Qué ofrece el ecosistema de Pydantic y por qué usarlo
Cuatro caminos	Los approaches del framework para construir agentes
Harness y dominio	El sistema que rodea al modelo y la mentalidad de agentes de dominio
Demo y producción	Agentes resolviendo un trabajo concreto y cómo operarlos

03 — La idea de partida

La frase con la que abre la charla viene de la documentación de Pydantic AI v1.0.0: *«We built Pydantic AI with one simple aim: to bring that FastAPI feeling to GenAI app and agent development»*. En español: «Construimos Pydantic AI con un solo objetivo: llevar esa sensación de FastAPI al desarrollo de aplicaciones y agentes de IA generativa».

«Esa sensación de FastAPI» es la experiencia de usar funciones normales de Python con tipos: el framework valida por ti, el editor te ayuda con el autocompletado, los errores son claros y hay poco código de pegamento. La promesa es que armar un agente se sienta igual de directo que armar una API.

04 — Agent loop: cómo decide un agente

Un agente es un modelo que elige qué herramienta usar hasta tener una respuesta. El ciclo es este:

1. La persona usuaria hace una petición.
2. El modelo la lee y decide si puede responder o si necesita una herramienta.
3. Si necesita una, la elige y la llama con argumentos.
4. La herramienta ejecuta y devuelve un resultado, que se suma al contexto.
5. El modelo vuelve a decidir con ese nuevo contexto.
6. Cuando tiene suficiente, produce la respuesta, que se valida antes de entregarla.

Lo importante es que el modelo **decide**, pero el código **ejecuta**. El modelo no corre nada por su cuenta: pide llamar a una función que tú escribiste y tú controlas lo que esa función puede hacer.

05 — El stack: tres piezas sobre tus modelos y tus datos

- **FastAPI** expone el agente como un endpoint y valida cada petición.
- **Pydantic** define el contrato: qué entra y qué sale, con tipos.
- **Pydantic AI** es el agente: el modelo, las herramientas y una salida tipada.
- **Modelos.** Más de 30 proveedores (OpenAI, Anthropic, Google Gemini y otros, también locales con Ollama o vLLM) se eligen con un texto como `"anthropic:claude-sonnet-5-5"`, o todos detrás de AI Gateway.
- **Datos.** Tu base de datos (PostgreSQL, MySQL, MongoDB, Redis, SQLite) no habla directo con el modelo: llega como una herramienta tipada que hace una consulta concreta, o como contexto que recuperas antes de llamar al agente.
- **Logfire**, del mismo equipo, muestra cada ejecución del agente: qué pidió, qué herramienta llamó y qué devolvió.

Las tres piezas comparten la misma base, los modelos de Pydantic. El mismo tipo que valida la petición describe la salida del agente y los argumentos de las herramientas.

06 — Un agente en código

El ejemplo de la charla es un agente que clasifica solicitudes de clientes de una empresa. Consulta un CRM de ejemplo para conocer el plan del cliente. Los datos son sintéticos y el código se probó con Pydantic AI 2.60.0 y FastAPI.

1. El contrato: una clase de Pydantic.

```
class Clasificacion(BaseModel):
    area: Literal["ventas", "soporte", "facturacion"]
    prioridad: Literal["alta", "media", "baja"]
    resumen: str
    requiere_humano: bool = Field(description="Si falta información")
```

`BaseModel` describe la forma exacta de la respuesta. `Literal` limita los valores: si el modelo inventa un área que no existe, la validación falla. `requiere_humano` es una salida honesta: en lugar de adivinar cuando falta información, el agente lo dice.

2. El agente y su herramienta.

```
agente = Agent(
    "anthropic:claude-sonnet-5-5",
    output_type=Clasificacion,
    instructions="Clasifica la solicitud. Usa el CRM.",
)

@agente.tool_plain
def consultar_crm(cliente_id: int) -> dict:
    """Devuelve la empresa y el plan del cliente."""
    return CRM.get(cliente_id, {"plan": "desconocido"})
```

`Agent` recibe el modelo como un texto: cambiar de proveedor es cambiar ese texto. `output_type` conecta el contrato y `instructions` orienta al modelo. `@agente.tool_plain` convierte una función de Python en herramienta. Su docstring y sus tipos le explican al modelo qué hace y qué argumentos recibe. El modelo pide la herramienta y tu código la ejecuta.

3. FastAPI lo convierte en un servicio.

```
class Solicitud(BaseModel):
    cliente_id: int
    mensaje: str

@app.post("/solicitudes")
async def clasificar(solicitud: Solicitud) -> Clasificacion:
    prompt = f"Cliente {solicitud.cliente_id}: {solicitud.mensaje}"
    resultado = await agente.run(prompt)
    return resultado.output
```

FastAPI valida la petición con Pydantic antes de llamar al agente: una petición sin `cliente_id` ni siquiera llega al modelo. `agente.run` corre el agent loop y `resultado.output` ya es una `Clasificacion` validada. El endpoint devuelve la misma clase, así que el contrato llega intacto al cliente.

07 — Qué ofrece Pydantic para construir agentes

La documentación de Pydantic AI organiza el ecosistema así:

Pieza	Para qué sirve
Pydantic	La base: validación y tipos
Pydantic AI	Agentes tipados: modelo, herramientas y salida estructurada
Pydantic AI Harness	Un paquete aparte (`pydantic-ai-harness`) con piezas listas: memoria, guardrails, subagentes, planificación y agentes completos como Coder o Researcher. Va en versión 0.x y su API puede cambiar
Pydantic Graph	Flujos de control explícitos, con estados y transiciones tipadas
Logfire	Observabilidad: trazas de cada ejecución
Pydantic Evals	Evaluación de la calidad de las salidas y de las llamadas a herramientas
AI Gateway	Una sola llave para muchos modelos, con control de costos

08 — Por qué Pydantic AI

Según su propia documentación («Why Pydantic AI»), estas son sus diferencias principales:

- **Cualquier modelo, una sola API.** Cambias de proveedor con un texto; ninguna función clave queda atada a un vendor.
- **Tipado de punta a punta.** Salida estructurada, dependencias y herramientas con tipos: el editor y el verificador de tipos saben qué devuelve el agente.
- **Medido, no a ojo.** Instrumentación con OpenTelemetry, trazas en Logfire con una línea y evaluaciones con Pydantic Evals.
- **Piezas componibles.** Las capabilities empaquetan herramientas, instrucciones, hooks y ajustes reutilizables; el Harness cubre el resto.

09 — Cuatro caminos para construir

1. **Agent en código.** `Agent(modelo, output_type=..., tools=...)`. El punto de partida para casi todo, y el que se mostró en la charla.
2. **Agent specs.** El mismo agente definido en YAML o JSON y cargado con `Agent.from_file("agente.yaml")`. Sirve para separar la configuración del código.
3. **Capabilities y Harness.** `capabilities=[...]` suma piezas listas al agente: búsqueda web, MCP, memoria, guardrails o subagentes.
4. **Pydantic Graph.** Estados y transiciones tipadas para cuando el flujo de control es el problema. La documentación es clara: para la mayoría de los agentes no hace falta, y conviene probar antes los caminos más simples.

La regla práctica: empieza por el camino 1 y sube solo cuando lo necesites.

10 — El harness: el sistema que rodea al modelo

El modelo predice texto; el harness es todo lo demás. Es el sistema que rodea al modelo y lo vuelve controlable:

- **Contexto.** Las instrucciones y los datos que recibe el modelo en cada paso.

- **Herramientas.** Las funciones de Python que puede pedir ejecutar.
- **Agent loop.** El ciclo que pide, ejecuta y vuelve a decidir.
- **Validación.** La salida se revisa con tipos de Pydantic antes de entregarla.
- **Límites.** Plazos, reintentos y cuántas llamadas puede hacer.
- **Registro.** Trazas de cada ejecución en Logfire.

El proveedor del modelo (OpenAI, Anthropic u otro) queda fuera y se llama por API. Por eso cambiar de modelo no cambia el harness: el contrato, las herramientas y las trazas siguen igual. Pydantic AI Harness es el paquete oficial con piezas listas para este sistema; va en versión 0.x.

11 — Producción: medir antes de escalar

1. **Pruebas.** `TestModel` reemplaza al modelo real: verificas la lógica del agente y del endpoint sin red y sin costo.
2. **Evaluaciones.** Pydantic Evals mide la calidad con casos conocidos, antes de cambiar el modelo, las instrucciones o las herramientas.
3. **Trazas.** Logfire muestra cada ejecución, cada herramienta y cada validación. Sin trazas, un fallo es una anécdota; con trazas, es un dato.
4. **Modelos y costo.** AI Gateway centraliza el acceso a varios modelos con control de costos.

Para ejecuciones largas que deben sobrevivir a una caída o a un reinicio existe durable execution, con motores como Temporal, DBOS o Prefect. Los sistemas multiagente y los grafos complejos quedan como lectura posterior.

12 — Agentes de dominio: mi mentalidad

La mentalidad que defiende la charla se dice en una línea: **no diseñes un agente para todo, diseña uno excelente para algo.**

Un agente de dominio hace un trabajo concreto. Recibe un contexto acotado, usa herramientas acotadas, respeta una política local y, cuando le falta información, pasa a revisión humana en lugar de adivinar. Un agente genérico intenta hacer de todo y por eso es difícil saber cuándo falla.

La referencia es la charla de Justin Schroeder (Standard Agents), «The Future Is Domain-Specific Agents», presentada en el AI Engineer World's Fair 2026. Es una visión, no un benchmark: no demuestra que un agente de dominio sea mejor en todos los casos.

13 — Por qué importan los agentes de dominio

Tres razones prácticas:

- **Se pueden evaluar.** Un trabajo concreto tiene entradas y salidas esperadas. Puedes juntar casos conocidos y medir si el agente lo hizo bien. Con un agente que «hace de todo», no sabes qué medir.
- **Se pueden acotar.** Si el contexto, las herramientas y la política son limitados, se reduce lo que puede salir mal. Un agente que solo puede consultar el plan de un cliente no puede borrar la base de datos.
- **Se pueden operar.** Con límites, trazas y revisión humana cuando falta información, el agente crece con control: sabes cuánto cuesta, cuánto tarda y cuándo pide ayuda.

14 — Por qué construir agentes hoy

- **El límite ya no es el modelo.** Los modelos ya eligen y llaman herramientas; lo que falta casi siempre es el sistema que los rodea.
- **El valor está en el trabajo.** Un agente que resuelve un proceso del negocio de punta a punta vale más que otro chat.
- **El harness es la ventaja.** Quien domina el contrato, los límites y las trazas controla el costo y el riesgo de lo que construye.

15 — La demostración: un trabajo concreto

La demostración muestra agentes de dominio resolviendo un trabajo concreto, con datos sintéticos. Para seguirla conviene mirar tres cosas:

1. **La entrada.** Qué recibe el agente y qué contexto se le da.
2. **La herramienta.** Qué función llama y con qué argumentos.
3. **La salida validada.** Qué forma tiene la respuesta y qué pasa cuando no se puede decidir.

16 — Las tres ideas que deben permanecer

1. **Los tipos son el contrato.** Pydantic valida la petición, los argumentos de las herramientas y la salida del agente.
2. **El harness hace fiable al modelo.** Contexto, validación, límites y registro rodean al modelo.
3. **Un agente, un trabajo.** Con contexto y herramientas acotados, el resultado se puede evaluar.

Una advertencia que conviene repetir: validar la forma de una salida no prueba que sea verdadera. Una respuesta puede cumplir el contrato de tipos y estar equivocada. Para eso existen las pruebas, las evaluaciones y la revisión.

17 — Glosario mínimo

Término	Significado
Agente	Un modelo que elige qué herramienta usar hasta tener una respuesta
Agent loop	El ciclo de decidir, actuar con una herramienta y volver a decidir
Harness	El sistema que rodea al modelo: contexto, herramientas, validación, límites y registro
Herramienta (tool)	Una función de Python que el modelo puede pedir ejecutar
Capability	Una pieza reutilizable de comportamiento que se suma a un agente con `capabilities=[...]`
`output_type`	El tipo que describe la forma de la respuesta del agente
Agente de dominio	Un agente diseñado para un trabajo concreto, con contexto y herramientas acotados
Traza	El registro de una ejecución: qué se llamó, con qué datos y qué devolvió
Evaluación	Una medición de calidad con casos conocidos

18 — Cómo practicar: cuatro pasos

1. **Lee los recursos.** Empieza por la documentación de Pydantic AI y las lecturas de esta guía. Con una hora de lectura alcanza para entender el modelo mental.
2. **Elige un dominio.** Escoge un trabajo concreto que conozcas bien: clasificar tickets, extraer datos de documentos, responder preguntas sobre un manual. Define su contexto, sus herramientas y cómo sabrás que salió bien.
3. **Mira lo que ocurre.** Activa trazas y revisa cada ejecución. Se aprende más de diez trazas reales que de cualquier teoría.
4. **Prueba.** Escribe una prueba de tu agente con `TestModel`, el modelo de pruebas de Pydantic AI: verifica la salida y la lógica sin llamar a un modelo real, sin red y sin costo.

19 — Cursos sugeridos para profundizar

Escogí cursos que se pueden empezar gratis y que cubren partes distintas del camino. Verifiqué sus páginas el 22 de octubre de 2026; precios y contenidos pueden cambiar.

Curso	Qué aprenderás	Nivel y costo
Pydantic for LLM Workflows (DeepLearning.AI, Ryan Keenan)	Modelos de Pydantic, validación de respuestas de modelos, pasar un modelo en una llamada a la API y llamadas a herramientas, con un proyecto de asistente de soporte. Es la base de «los tipos son el contrato».	Intermedio, unas 2 horas. Gratis para inscribirse; el reconocimiento requiere membresía Pro.
Agentic AI (DeepLearning.AI, Andrew Ng)	Flujos agénticos, reflexión, uso de herramientas y MCP, evaluaciones y análisis de errores, y patrones de agentes muy autónomos. Da el marco conceptual para decidir cuánta autonomía darle a un agente.	Intermedio, unas 10 horas. Gratis para auditar; el certificado requiere membresía Pro.
AI Agents Course (Hugging Face)	Teoría, diseño y práctica de agentes con smolagents, LlamaIndex y LangGraph, y unidades extra de observabilidad y evaluación. Sirve para comparar enfoques de otros frameworks con el de Pydantic AI.	Python básico. Gratis, con dos certificados gratuitos; unas cinco semanas a ritmo de 3 a 4 horas por semana.
AI Agents for Beginners (Microsoft, GitHub)	18 lecciones con código para empezar a construir agentes. Una alternativa práctica si prefieres leer y ejecutar código a ver videos.	Principiante. Gratis, en GitHub.

20 — Lecturas y documentación recomendadas

Cada recurso con lo que vas a encontrar y para qué sirve:

- **Documentación de Pydantic AI.** El punto de partida. Explica el agente, las herramientas y la salida estructurada. Lee primero la página de salida (`output_type`), que es el contrato de la charla.
- **Pruebas unitarias con Pydantic AI.** Cómo usar `TestModel` y `override` para probar un agente sin llamar al modelo. Es el cuarto paso de la sección anterior.
- **Integración con Logfire.** Cómo ver trazas de cada ejecución de un agente. Útil para el tercer paso.
- **Tutorial de FastAPI.** Si todavía no conoces FastAPI, este tutorial explica la validación con Pydantic y las rutas que usa el endpoint de la charla.

- **Pydantic AI Harness.** Las piezas oficiales para agentes complejos: memoria, guardrails, subagentes y agentes completos.
- **Agent specs.** Cómo definir un agente en YAML o JSON.
- **«Building effective agents» (Anthropic).** Un texto que distingue flujos de trabajo de agentes y presenta patrones como encadenar prompts, enrutar y orquestadores con trabajadores. Su consejo central: empieza por la solución más simple que funcione y agrega complejidad solo cuando mejora algo medible.
- **«Pydantic AI v1» (Samuel Colvin).** El anuncio de la versión 1: qué promete el framework y por qué le importa la estabilidad.
- **«The Future Is Domain-Specific Agents» (Justin Schroeder).** La charla que inspira la parte de agentes de dominio.

21 — Temas para profundizar

- **Historial de mensajes.** Guardar la conversación de un agente para continuar donde se quedó.
- **Recuperación de información.** Dar al agente una herramienta que busque en tu propio contenido para responder con tus datos.
- **Evaluaciones.** Medir la calidad con casos conocidos antes de cambiar el modelo, las instrucciones o las herramientas.
- **Trazas con Logfire.** Ver cada ejecución, cada herramienta y cada validación.
- **Reintentos con `ModelRetry`.** Qué hacer cuando una validación de salida falla: pedirle al modelo que lo intente de nuevo con el error a la vista.
- **Capabilities y Harness.** Sumar memoria, guardrails o subagentes con piezas oficiales.
- **Durable execution.** Ejecuciones que sobreviven a caídas, con Temporal, DBOS o Prefect.

22 — Fuentes y límites de lo dicho

- El código mostrado es de ejemplo, con datos sintéticos, y se probó con Pydantic AI 2.60.0 y FastAPI al prepararlo. La API de Pydantic AI cambia entre versiones: revisa la documentación de la versión que instales. Los ejemplos usan `output_type`, no la API antigua `result_type`.
- Los sistemas multiagente y Pydantic Graph solo se mencionaron. La descripción del ecosistema viene de la documentación de Pydantic AI, consultada el 22 de octubre de 2026.
- La visión de agentes de dominio es una postura, respaldada por una charla, no por un estudio comparativo.
- Los logos y nombres de herramientas pertenecen a sus titulares y se mencionan solo como referencia.

23 — Sobre el expositor

Ricardo Tovar. AI Engineering en formación en la Universidad Tecnológica de Panamá y cofundador de MART Automations. Su interés es llevar agentes de la demostración a producción con la menor magia posible.

- GitHub: github.com/rtovardev
- LinkedIn: linkedin.com/in/ricardotovar-ai
- Instagram: instagram.com/ricardotovar.ai
- Tarjeta digital: ricardo-card.vercel.app